Automated Label Inspection System

A three-month research study of text localization, data curation and synthetic print-defect classification

Harsh Chauhan

Research project report · 2 November 2026

Repository: `harsh-21-chauhan/Label_Inspection` Evidence revision: `ab9431f7e643700cc325be02d2ce80e2c847db1e`

Abstract

This project investigates how to identify defective printed text on product labels when authentic defect data are scarce and text appears at different scales, orientations and surface curvatures. The research progressed from OpenCV preprocessing and Tesseract OCR to geometric alignment with SIFT, morphological defect synthesis, ShuffleNetV2 classification, Mask R-CNN text detection and a custom DBNet implementation. Dataset preparation combined CLIP-based filtering of CTW1500 and Total-Text with web scraping and EasyOCR-generated annotations. The final saved detection manifests contain 270 training images with 3,475 text annotations and 63 test images with 1,259 annotations. A separate classification dataset contains 300 original text crops and three synthetic defects per crop, giving 1,200 patches.

The repository demonstrates a substantial research prototype, including data preparation, model definitions, training code and qualitative predictions. An independent audit reproduced the data path and architecture execution, measured 12,342,210 parameters in DBNet and 1,255,654 in ShuffleNetV2, and evaluated the classical localization baseline on all 63 test images. Its mean pixel intersection over union was 0.1671. Saved learned-model outputs show both successful predictions and errors. However, trained checkpoints and numerical evaluation logs are absent from this revision; validation crops also share source images with training crops, and some scraped images are unrelated educational diagrams. Accordingly, this report separates verified measurements from qualitative evidence and proposed integration. The work supports a defensible study of an inspection pipeline, while stronger performance claims require an independent, domain-correct evaluation.

Keywords: label inspection; synthetic defects; CLIP curation; text detection; DBNet; ShuffleNetV2; computer vision.

Main verified quantities

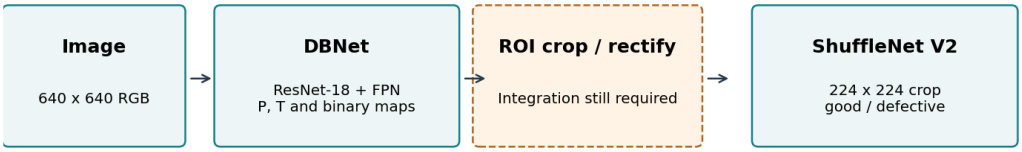
Quantity	Evidence-backed value
Final detection data	270 training / 63 test images
Final text annotations	3,475 training / 1,259 test regions
Classification data	960 training / 240 validation patches
Classical baseline, all 63 test images	Mean pixel IoU 0.1671
Saved learned-model artifacts	30 detection overlays; 65 individual classification overlays
Learned-model accuracy and DBNet IoU	Not independently reproducible from this commit

1. Problem definition and research progression

Product-label inspection involves several distinct tasks. **Detection** locates printed text; **classification** judges whether a crop is normal or defective; **recognition** converts text pixels into characters. The industrial objective is to find missing strokes, blurred print or merged characters without rejecting acceptable labels. Curved packaging, reflections, small text and limited authentic defects make this harder than classifying a clean, tightly cropped image.

The central contribution of this three-month project is the experimental progression: identifying a limitation, introducing a more suitable component and examining what remains unresolved. The stages are not a controlled performance ranking because identical test sets, checkpoints and logs were not retained for every method.

Stage and repository implementation	Limitation identified	Response and remaining limitation
Grayscale, CLAHE, Otsu thresholding and morphology	Fixed image operations cannot reliably separate all label text from graphics and background	Adaptive localization improves proposals, but still uses hand-tuned area and aspect-ratio rules
Tesseract, including OCR on localized regions	Early trials were reported as unsuccessful; damaged or poorly localized text remains difficult	Separate text localization from downstream interpretation; no OCR benchmark proves a universal failure
SIFT, FLANN and RANSAC alignment	Appearance varies between template and target views	Align corresponding views before comparison; unrelated labels and nonplanar distortion remain difficult
Morphological defect synthesis	Too few authentic defective examples	Create controlled variants; synthetic changes do not establish real-defect realism
ShuffleNetV2 on label/text images	A classifier cannot locate its own region of interest	Train on explicit crops and add a text detector; crop quality still governs classification
Mask R-CNN, then custom DBNet	General instance segmentation adds proposal and mask machinery	Use dense text-oriented prediction; an actual speed/accuracy comparison remains unmeasured



The classifier predicts print quality. Reading characters requires a separate OCR recognizer.

Figure 1. Intended text-aware inspection architecture. The detector and classifier exist as separate modules; the crop/rectification connection is not implemented as an end-to-end inspection application in this revision.

ShuffleNetV2 in `model.py` ends in a two-class linear layer. It does **not** extract character sequences. DBNet likewise predicts text-region maps rather than transcriptions. Tesseract is the implemented recognition baseline; a dedicated recognizer would be an additional component if expiry dates, batch numbers or label wording must be read. The final output envisaged here is print-quality classification, not verified semantic correctness.

2. Dataset construction and curation

2.1 CLIP-based relevance filtering

The project uses normalized CLIP ViT-B/32 image and text embeddings [6]. An image is compared with positive packaging prompts and negative prompts describing signs, storefronts, documents and other undesired content. If v is the image embedding, and p and n denote positive and negative prompt embeddings, the saved score is a difference of mean cosine similarities:

$$s(I) = \frac{1}{N_+}\sum_i v^T p_i - \frac{1}{N_-}\sum_j v^T n_j; \quad \text{keep} \iff s(I) > 0.020$$

This is a semantic relevance score, not a defect probability or a calibrated label-quality score. The CTW1500 CSV records the earlier filter; the current `clip_filter_dataset.py` defaults to Total-Text and uses revised prompts. They should not be described as one identical prompt configuration.

Source and split	Images scored	Marked as kept in CSV	Retained percentage
CTW1500 training	1,000	150	15.00%
CTW1500 test	500	17	3.40%
Total-Text training	1,255	194	15.46%
Total-Text test	300	54	18.00%
Combined	3,055	415	13.58%

These are counts from the score files, not a fresh execution of CLIP. The current filtered training folders contain 108 of the 150 CTW1500 entries and 166 of the 194 Total-Text entries; the older annotation JSON files still contain the removed entries. `merge_datasets.py` synchronizes by file presence and avoids including those missing files in the merged manifest.

2.2 Scraping, weak annotations and final manifests

`colab_label_scraper_and_annotator.py` specifies ten search queries, up to 25 candidates per query, minimum download dimensions of 400 × 400 and an extreme-aspect-ratio rejection rule. Despite the file's Google wording, its crawler is `BingImageCrawler`. EasyOCR detections with confidence at least 0.15 are retained, and an image must have at least two detections. The saved export contains 77 images and 1,514 text regions. These are **automatic annotations**, not independently checked ground truth; the script's confidence threshold is permissive.

Final manifest	CTW1500	Total-Text	Web-scraped	Total images	Text annotations
`train_bottles_curated.json`	95	113	62	270	3,475
`test_bottles_curated.json`	15	33	15	63	1,259
Combined	110	146	77	333	4,734

All final manifest images exist and have matching recorded dimensions. No byte-identical or decoded-pixel-identical image was found across the final detection train/test manifests. This does not rule out near duplicates or related views. The final curated JSON files are provided, but the step producing them from `train_merged.json` and `test_merged.json` is not captured in a dedicated script.

3. Classical preprocessing, OCR and geometric alignment

3.1 OpenCV preprocessing and localization

`preprocessing.py` converts an RGB/BGR image to grayscale, applies CLAHE with clip limit 2.0 and an 8 × 8 tile grid, computes an Otsu binary threshold and applies a 3 × 3 morphological closing. Grayscale reduces the channel dimension; CLAHE increases local contrast; thresholding separates bright and dark structures; closing joins nearby foreground regions. Their effect depends on print polarity and scale. Removing colour and thresholding can also remove faint strokes or merge characters with label graphics.

`part2_localization.py` uses a 5 × 5 Gaussian blur, adaptive Gaussian inverse thresholding with block size 15 and constant 5, followed by a 15 × 5 rectangular closing. It retains contour bounding boxes only when 300 < width × height < 50,000 and width/height > 1.2. These absolute pixel thresholds explain why resizing, decorative graphics and vertical text can change its behaviour substantially.



Figure 2. Preprocessing reproduced during the audit on `ctw_1097.jpg`, using the settings from `preprocessing.py`. This illustration is not a trained-model prediction.

3.2 Tesseract OCR

`ocr.py` applies Tesseract to the preprocessed image. `part2_ocr.py` instead crops classical localization proposals, adds five pixels of padding, uses Otsu thresholding and runs `--psm 7`, which assumes a single text line. This improves task focus when a proposal contains one line; a multi-line or incorrectly cropped region violates that assumption. Curved, stylized and defective text can also remain unreadable.

The project history reports unsuccessful OCR trials. Since no paired transcriptions, character error rates or complete OCR logs are saved, the defensible conclusion is that these trials did not meet the project's needs—not that Tesseract fails on every label. The detector/classifier alternative addresses localization and print-quality judgment; it does not solve character recognition automatically.

3.3 SIFT alignment

`part2_sift_alignment.py` computes SIFT descriptors [1], matches them with FLANN using two nearest neighbours and applies Lowe's ratio test with threshold 0.7. With more than four accepted matches, RANSAC estimates a homography using a five-pixel reprojection threshold:

$$\lambda[x', y', 1]^T = H[x, y, 1]^T$$

The code uses a projective homography and `warpPerspective`, despite comments describing an affine transform. It requires corresponding label content; unrelated products or strong cylindrical distortion can defeat a single homography. A failed homography is not checked before warping. Alignment controls pose, while synthesis varies appearance. No registration-error benchmark is saved.

4. Addressing defect scarcity through synthesis

Morphological synthesis creates known changes without requiring naturally defective production samples. For bright-background, dark-text images, dilation selects local maxima and tends to shrink dark strokes; erosion selects local minima and tends to expand them. The interpretation reverses for bright text on a dark background. Thus the operation name alone does not guarantee a particular physical print defect.

$$(I \oplus K)(x) = \max_{u \in K} I(x-u), \qquad (I \ominus K)(x) = \min_{u \in K} I(x+u)$$

Three different expansion schemes appear in this repository and must be distinguished:

Implementation	Generated variants	Expansion relative to originals
`part2_defect_synthesis.py`	Missing: 7 × 7 dilation twice; “blur”: 3 × 3 dilation; adhesion: 4 × 4 opening	Three defective variants; four images if the original is retained
`part3_data_augmentation.py`	Original, masked light dilation and masked heavy dilation	Three images per original
`generate_defect_dataset.py`	Normal crop; 5 × 5 dilation; 3 × 3 dilation plus Gaussian blur; 4 × 4 erosion	Four patches per original crop

The masked scheme preserves pixels outside the annotated text mask:

$$I_{aug} = M \odot f(I) + (1 - M) \odot I$$

Masking preserves the background, but the code fills Bezier control points as polygon vertices and covers text regions rather than individual ink strokes. The final DBNet loader uses another scheme: clean, photometric/blur and random morphological variants. It changes whole images; the earlier masking advantage is therefore absent.



Figure 3. Four existing classification patches from crop group `0020`: normal, synthetic missing print, blur and adhesion. All are generated from the same underlying crop and are not four independent label observations.

The classification generator shuffles annotations with NumPy seed 42, takes up to 300 usable boxes, pads each by 10% of its width and height, and resizes to 224 × 224. Every fifth base crop enters validation; its three defective siblings stay in that split. This produces 240 normal + 720 defective training patches and 60 normal + 180 defective validation patches. The full dataset therefore contains 300 originals + 900 variants = 1,200 patches, not 1,200 independent real images.

Fixed kernels, resizing and print polarity can create synthetic shortcuts. Testing on authentic defects and acceptable low-contrast print is necessary to distinguish learned print quality from recognition of the generator's signature.

5. Text detection: Mask R-CNN to DBNet

5.1 Mask R-CNN exploration

The Mask R-CNN stage [3] uses a ResNet-50/FPN detector with a region-proposal network, ROI features, box classification/regression and an instance-mask head. The project replaces its predictors with two-class heads for background and text. This supplies an appropriate conceptual response to ShuffleNetV2's lack of spatial localization, including the possibility of irregular text masks.

The saved dataset class restricts the experiment to the first five source images and their three variants: 15 samples. The training script runs one epoch, does not save a checkpoint and refers to the older raw CTW1500 directory. Consequently, this is an exploratory implementation, not a validated detector benchmark. The model factory uses `weights=None`; torchvision may still initialize the backbone with its default ImageNet weights. This is different from loading a text-trained or COCO detection checkpoint. Neither the slides' speed comparisons nor an assertion that Mask R-CNN “failed because it was slow” is established by a saved timing experiment.

5.2 Custom DBNet architecture

The custom DBNet [4] uses ResNet-18 feature maps at strides 4, 8, 16 and 32. Lateral 1 × 1 convolutions map them to 256 channels, top-down additions fuse information, and 3 × 3 convolutions reduce each level to 64 channels. All four are resized to stride 4 and concatenated into 256 channels. Two upsampling heads then predict a probability map P and threshold map T at the input resolution. Its differentiable binary map is:

$$B_{ij} = \frac{1}{1 + \exp[-50(P_{ij} - T_{ij})]}$$

P estimates text presence; T provides a local threshold; the steep sigmoid permits gradients during training. A 640 × 640 input produced three tensors of shape [1, 1, 640, 640] during the audit. This confirms executable architecture and data compatibility, not successful text detection.

Model, with project output heads	Parameter count measured from code	Role
Mask R-CNN, ResNet-50/FPN, two classes	43,975,515	General text-instance localization prototype
Custom DBNet, ResNet-18/FPN	12,342,210	Dense text-region prediction
ShuffleNetV2 x1.0, two classes	1,255,654	Crop-level normal/defective classification

DBNet has 71.93% fewer parameters than the two-class Mask R-CNN configuration. This is a measured model-size comparison. It does not imply a corresponding latency reduction: input resolution, operations, hardware, postprocessing and batch size all matter. No verified DBNet FPS result is available in this commit.

6. DBNet training targets, loss and inference

The final training entry point reads `train_bottles_curated.json`, uses 640 × 640 RGB inputs scaled to [0, 1], and enables ImageNet initialization for ResNet-18. The configured optimizer is Adam with learning rate 0.001 and weight decay 0.0001. The command-line default is five epochs with batch size four; the Python function default is ten epochs. Actual historical training duration cannot be recovered without the run log. Device selection prefers Apple MPS, then CUDA, then CPU.

For each text region, `DBNetDataset` fills the **full polygon** as the probability target Y. It computes an inward offset using shrink ratio r = 0.4, polygon area A and perimeter L:

$$d = \frac{A(1 - r^2)}{L + 10^{-6}}$$

The threshold target Q is 0.7 inside the full polygon and 0.3 inside its shrunken interior; its supervision mask M covers the full polygon. The probability target is not the shrunken kernel. Bezier control points are filled directly as polygon vertices, and geometric failures in shrink computation are silently skipped. These details make this a **simplified DBNet implementation**, rather than an exact reproduction of the original paper's target generation, balanced losses and postprocessing.

The implemented objective is:

$$\mathcal{L} = \mathrm{BCE}(P, Y) + \mathrm{BCE}(B, Y) + 10\,\mathrm{mean}(|M \odot (T - Q)|)$$

The L1 term is averaged over the entire image after masking, rather than divided by the number of active mask pixels. Its effective strength therefore varies with text coverage. The architecture/data-path audit produced finite loss for a real dataset sample using untrained weights; this is a compatibility check, not a convergence result. The loader also substitutes a black image when a file is missing, which could conceal data errors. All final manifest files were present during this audit.

The saved test script applies its own heuristic postprocessing. It smooths P with a 5 × 5 Gaussian kernel and selects a threshold from the map's mean and standard deviation:

$$\tau = \max(0.40, \mu_{P_s} + 0.8\sigma_{P_s}), \qquad \widehat{M} = \mathrm{close}_{7\times 3}\,(P_s > \tau)$$

The mask is resized with nearest-neighbour interpolation; only contours with area above 80 original-image pixels are drawn. T and B are computed but not used for the final threshold. Moreover, IoU is computed from the entire closed mask, while tiny contours are excluded from the visualization. A clean-looking overlay therefore need not correspond exactly to the scored mask.

`part5_dbnet_test.py` defaults to the first 15 manifest entries, which are the scraped educational diagrams in this revision. It also falls back to random head weights when the checkpoint is missing. Because that branch can still create output images, neither an image filename nor a coloured heatmap proves that a trained model was loaded. A future evaluation should fail explicitly when the checkpoint is unavailable and save the checkpoint hash, complete image list, settings and predictions.

7. ShuffleNetV2 classification and validation design

ShuffleNetV2 x1.0 [2] provides a compact crop classifier using channel splitting, pointwise/depthwise operations and channel shuffling. The final fully connected layer is replaced by a two-class head: 0 = normal and 1 = defective. All three synthetic defect types map to the same defective class; this model does not identify the defect subtype as a separate output.

Training starts from ImageNet weights. `dataset.py` resizes crops to 224 × 224 and normalizes channels with ImageNet means [0.485, 0.456, 0.406] and standard deviations [0.229, 0.224, 0.225]. Training additionally uses horizontal flips with probability 0.3 and brightness/contrast jitter of 0.15. Mirrored text is a robustness perturbation rather than a typical manufacturing defect; its benefit should be tested. Validation transformations are deterministic.

`train.py` configures SGD with learning rate 0.01, momentum 0.9 and weight decay 0.00005, plus a cosine learning-rate schedule. The default is ten epochs with batch size 16, and the checkpoint with the highest validation accuracy is saved. These are code settings, not confirmation of which settings produced the retained overlays.

To address the normal:defective ratio of 1:3, inverse-frequency class weights are computed from the training split:

$$w_c = \frac{N}{Cn_c}; \qquad w_{normal} = \frac{960}{2(240)} = 2, \quad w_{defect} = \frac{960}{2(720)} = \frac{2}{3}$$

These weights enter weighted cross-entropy, giving the two classes equal total nominal weight. Always predicting “defective” would already achieve 75% accuracy on this synthetic validation distribution; accuracy must therefore be accompanied by defect recall and normal specificity.

7.1 Parent-image overlap

The generator keeps four siblings of each base crop in the same split, avoiding the simplest direct augmentation leak. However, it splits **crops**, not parent images. Replaying its order recovered all 300 crop identities: 299 generated normal JPEGs matched byte-for-byte, and the remaining crop differed by a mean of less than 0.0005 intensity units on a 0–255 scale.

Reconstructed split property	Result
Distinct parent images contributing training crops	130
Distinct parent images contributing validation crops	42
Parent images appearing in both	29
Validation base crops with a training parent	46 / 60 = 76.67%
Corresponding validation patches, including variants	184 / 240 = 76.67%

This does not show that the exact validation patch was used for training. It shows that the same label image, background, typography or product identity can appear through different crops. Validation is also used to select the checkpoint, so it is not an untouched final test set. Split parent images or product groups first, generate crops and defects within each split, and reserve a separate final test set before claiming generalization to unseen products.

8. Quantitative and qualitative results

8.1 New classical-baseline measurement

The audit executed `part2_localization.localize_text` on **all 63 final test images**. Its proposed rectangles were combined into a foreground mask and compared with the union of annotation masks. To match the existing repository convention, Bezier control points were rasterized as polygon vertices. The metric is pixel-area IoU, not official CTW1500/Total-Text instance detection scoring:

$$\mathrm{IoU}(M, G) = \frac{|M \cap G|}{|M \cup G|}, \qquad \mathrm{mIoU} = \frac{1}{n} \sum_{i=1}^{n} \mathrm{IoU}(M_i, G_i)$$

Test subset	Images evaluated	Mean image pixel IoU
CTW1500-labelled subset	15	0.0358
Total-Text-labelled subset	33	0.1053
Web-scraped-labelled subset	15	0.4346
Entire saved test manifest	63	0.1671

The 48 CTW1500/Total-Text entries together yield mean IoU 0.0836. The scraped subset scores much higher, but all 15 are educational energy diagrams. Their layouts differ from product-label imagery; the pooled score therefore hides a meaningful domain difference. These results support moving beyond fixed contour heuristics. They do not quantify how much DBNet improves on this baseline.

8.2 What the retained learned-model outputs establish

There are 30 saved DBNet detection overlays and 30 heatmaps. Only 28 of those image identities are in the current 63-image test manifest; `ctw_1046.jpg` and `ctw_1084.jpg` are absent from it. This suggests mixed evaluation histories. Some outputs outline portions of text; others miss visible text or show fragmented responses. Without the checkpoint and per-image scores, no defensible mean DBNet IoU can be reconstructed from colour overlays.

The classification folder contains 65 individual prediction overlays and one montage. The overlays were transcribed and the nine mismatches visually checked. The counts below concern only those retained images, which may combine random selections from different runs:

Filename-derived target	Overlay predicts normal	Overlay predicts defective	Saved examples
Normal	13	6	19
Synthetic defective	3	43	46
Total	16	49	65

Thus 56 saved examples agree with their labels and nine do not. This is a descriptive audit of selected artifacts—not the 240-patch validation confusion matrix. The three false passes show why a “works perfectly” claim would be inaccurate. Confidence percentages are softmax outputs, not calibrated probabilities of correctness.

9. Visual analysis of the retained predictions



Figure 4. Existing DBNet-labelled overlays: partial text outlining on `ctw_1017.jpg`, missed or sparse responses on `ctw_1130.jpg` and `ctw_1390.jpg`, and a scraped energy diagram. These are preserved repository artifacts; their trained-checkpoint provenance is unavailable.

The product-label examples show why visual inspection must consider missed regions as well as green contours. Localizing a few strong strokes can look convincing while omitting a brand name, small text or a curved line. Heatmaps indicate the model output's spatial intensity, but their colour scale is not an accuracy measure. A report should display successes and failures together, as above, and attach numerical per-image detection results when available.



Figure 5. Existing ShuffleNetV2-labelled overlays. The upper row illustrates correct saved decisions. The lower row shows a normal crop rejected as defective, a missing-print variant passed as normal and an adhesion variant passed as normal. Labels and confidence values are reproduced from the saved artifacts.

The classification examples demonstrate nontrivial behaviour: several synthetic defects are distinguished from their normal counterparts, but natural appearance variation can trigger false rejection and some morphological changes are missed. The lower-row missing-print example is close to a 50% softmax decision; another false rejection reaches 75.7%. Confidence alone cannot replace a calibrated decision rule and independent validation.

In the current evaluation, classifier inputs are annotation-derived crops generated in advance. They are not crops predicted by DBNet. Therefore the saved classifier decisions do not establish end-to-end image-to-inspection performance. A detector error changes both the content and scale of the crop; the classifier must be tested on those imperfect predicted regions before the two-module architecture can be called validated.

10. Limitations, alternatives and required improvements

10.1 Domain relevance is not guaranteed by file names

Visual review found that `bottle_label_0001.jpg` through `bottle_label_0020.jpg` are educational energy diagrams. The first 15 are the entire scraped portion of the final test manifest: 15/63 = 23.81% of test images. The remaining five also enter the scraped training portion. Additional scraped content includes general medicine illustrations, aluminium material images and charts. These findings establish contamination in this saved export, without implying a measured relevance precision for the complete dataset. The CLIP filtering stage applies to CTW1500 and Total-Text; the scraping script does not apply the same CLIP screen to its export.



Figure 6. Three actual scraped files whose names imply bottle labels but whose contents are educational energy diagrams. OCR acceptance establishes text presence, not industrial-label relevance.

Priority	Limitation in this repository	Specific correction and verification
1	Weights, complete predictions and run logs are not shared	Retain both trained checkpoints and their hashes; save actual arguments, seed, split hashes and per-sample predictions; prohibit missing-checkpoint fallback
2	Out-of-domain scraped images and weak labels	Review every scraped image for product-label relevance; correct text polygons independently; keep provenance and a reviewed relevance manifest
3	Crop-level validation shares parent images with training	Split by source image and, preferably, product identity before synthesis; reserve a separate final test set
4	Synthetic-only defect labels	Add authentic normal and defective print examples; test separately on real defects and on held-out synthetic perturbations
5	Simplified DBNet targets and heuristic scoring	Sample Bezier boundaries, validate masks visually, use a documented DBNet target/loss/postprocessor and measure instance precision/recall/F1
6	No validated detector-to-classifier integration	Crop and rectify predicted text regions, classify each crop and define region-to-label aggregation; score missed defects and false rejections per label

No code or output was deleted to conceal these limitations. They explain the next experiments and preserve the value of the progression: synthesis addresses sample availability, detection addresses region selection, and separate classification addresses print quality. None alone resolves dataset validity or real-world generalization.

11. Evaluation protocol and interpretation

For the next definitive run, use a reviewed, frozen test set that contains the intended product categories and authentic defects. Report detection metrics separately from classification metrics, and evaluate the combined system on full images. Every generated defect variant must remain in its parent's split; near-duplicate views and the same product design should be grouped where possible. Human checking is especially important for EasyOCR pseudo-labels: comparing a model to another model's omissions can reward incomplete detection.

With defective print defined as the positive class, report the following from a complete held-out confusion matrix:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{Specificity} = \frac{TN}{TN + FP}$$

$$F_1 = \frac{2TP}{2TP + FP + FN}, \quad F_2 = \frac{5TP}{5TP + 4FN + FP}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

`test_shufflenet.py` computes the standard precision, recall, specificity, F1 and F2 quantities, but its printed “false alarm rate” is 1 − precision. That quantity is the false discovery rate, FP/(TP + FP); the false positive rate is FP/(FP + TN). Defect-subtype recall is a useful breakdown, but it is not an ablation study. A true ablation compares controlled model or data changes, such as training with and without synthesis using the same split and evaluation protocol.

For detection, use one-to-one matching between predicted and independently reviewed text instances under a stated IoU rule; report precision, recall and F1 in addition to pixel overlap. For OCR, use verified transcriptions and character/word error rates. For industrial inspection, report false acceptance of defects and false rejection of good labels at the chosen operating threshold. These assess different tasks and cannot be substituted for one another.

Latency must identify hardware, backend, input size, batch size, warm-up and synchronization. The classifier script times a 224 × 224 model forward pass on a synthetic tensor; that excludes image loading, DBNet, crop extraction, multiple regions and aggregation. Its conversion from crop FPS to bottles/minute is therefore not a measured production-line capacity. CUDA benchmarking would also need explicit synchronization, which the current script only performs for MPS.

Conclusion and project contribution

This project provides an implemented study of the components needed for automated label inspection: classical image processing and localized OCR; SIFT-based registration; multiple defect-synthesis approaches; CLIP-assisted relevance filtering; an EasyOCR-annotated web-data pipeline; merged and curated detection manifests; a custom DBNet detector; and a compact ShuffleNetV2 defect classifier. The strongest currently reproducible contributions are the data-processing pipeline, concrete architecture implementations, controlled synthetic examples and the analysis of why crop-aware inspection is needed.

The results are suitable for an academic **research-prototype report** when presented with the evidence boundaries above. They do not yet establish high-accuracy, real-time industrial deployment or perfect operation on arbitrary images. For a CV, a defensible description is: “Developed a label-inspection research pipeline combining CLIP-assisted dataset curation, synthetic print-defect generation, DBNet text detection and ShuffleNetV2 classification; investigated localization failures and validation leakage.” Add numerical learned-model performance only after a reproducible, independent evaluation.

References

[1] D. G. Lowe. “Distinctive Image Features from Scale-Invariant Keypoints.” International Journal of Computer Vision, 2004. DOI: 10.1023/B:VISI.0000029664.99615.94.

[2] N. Ma, X. Zhang, H.-T. Zheng and J. Sun. “ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design.” ECCV, 2018. arXiv:1807.11164.

[3] K. He, G. Gkioxari, P. Dollár and R. Girshick. “Mask R-CNN.” ICCV, 2017. arXiv:1703.06870.

[4] M. Liao, Z. Wan, C. Yao, K. Chen and X. Bai. “Real-Time Scene Text Detection with Differentiable Binarization.” AAAI, 2020. arXiv:1911.08947.

[5] R. Smith. “An Overview of the Tesseract OCR Engine.” ICDAR, 2007.

[6] A. Radford et al. “Learning Transferable Visual Models From Natural Language Supervision.” ICML, 2021. arXiv:2103.00020.

[7] CTW1500 dataset and implementation resources: `github.com/Yuliang-Liu/Curve-Text-Detector`. Total-Text: C. K. Ch'ng and C. S. Chan, “Total-Text: A Comprehensive Dataset for Scene Text Detection and Recognition,” ICDAR, 2017.

[8] EasyOCR project: `github.com/JaidedAI/EasyOCR`. Torchvision model implementations and pretrained-weight APIs: `pytorch.org/vision/0.20/models.html`.

[9] H. Chauhan. `harsh-21-chauhan/Label_Inspection`, revision `ab9431f`, and the supplied 13-slide “Automated Label Inspection System” presentation. These are the primary project-specific sources; slide-level speed and accuracy claims were not treated as measured evidence.

Appendix A. Evidence and reproduction

The report audit used Python 3.12.14, PyTorch 2.5.1+cpu, torchvision 0.20.1+cpu and OpenCV 4.11.0 on CPU. It did not train a replacement model or present randomly initialized predictions as learned results. The architecture checks used `pretrained=False`, and the Mask R-CNN parameter count explicitly disabled backbone downloads before installing the project's two-class heads. All 140 saved output files matched their recorded checksums after the audit.

The accompanying `evidence/audit.json` records dataset counts, missing-file checks, parent overlap, source revision, architecture checks and output hashes. `crop_provenance.csv` records the reconstructed source of each of the 300 crops; `classical_baseline_per_image.csv` records all 63 new baseline scores. `saved_overlay_transcriptions.json` records the 65 retained classifier labels. The baseline uses the repository's approximate mask convention; its values should not be compared directly with official text-detection leaderboard scores.

Reproduce the read-only audit from the repository root in an environment containing the project dependencies:

```
python docs/report/evidence/reproduce_audit.py
python docs/report/evidence/reproduce_figures.py
```

These scripts write only report evidence/assets and do not change application source, training data or original outputs. Their architecture tests confirm execution and shapes, not training quality. The report remains limited to the supplied revision: a newer checkpoint or log can change the performance assessment only after it is inspected and matched to its split.

Appendix B. Repository-specific source map

All 23 root-level Python modules and the supplied scraping notebook were inspected. Earlier experimental files remain part of the research history; they should not be deleted merely because a later stage supersedes their default workflow.

Research component	Source files inspected	Evidence or qualification
Input and preprocessing	`read_image.py`, `preprocessing.py`	Fixed example path; interactive visualization; grayscale, CLAHE, threshold and closing
Classical localization and recognition	`ocr.py`, `part2_localization.py`, `part2_ocr.py`	Tesseract baseline and contour proposals; no saved OCR accuracy benchmark
Registration	`part2_sift_alignment.py`	SIFT/FLANN ratio matching, homography and perspective warp; no saved registration error
Early and masked synthesis	`part2_defect_synthesis.py`, `part3_data_augmentation.py`	Three standalone defects versus two masked variants plus original
Relevance filtering	`clip_filter_dataset.py`, both `all_scores.csv` files	Current Total-Text configuration and historical CTW1500 score evidence
Web data acquisition	`colab_label_scraper_and_annotator.py`, matching `.ipynb`	Bing crawler and EasyOCR weak labels; saved content includes irrelevant images
Dataset synchronization	`merge_datasets.py`, merged and curated annotation JSONs	Merge follows existing files; final bottle-curation step has no dedicated script
Crop generation	`generate_defect_dataset.py`	Seed 42, 300 crops, four siblings, crop-level 80/20 split
Classification	`dataset.py`, `model.py`, `train.py`, `test_shufflenet.py`	Binary ShuffleNetV2, weighted loss, validation selection and retained overlays
Mask R-CNN prototype	`part4_text_detection_dataset.py`, `part4_text_detection_model.py`, `part4_text_detection_train.py`	First five images, three variants, one epoch; old raw-data paths
DBNet	`part5_dbnet_dataset.py`, `part5_dbnet_model.py`, `part5_dbnet_train.py`, `part5_dbnet_test.py`	ResNet-18/FPN, simplified targets, dynamic augmentation and heuristic test threshold
Historical output artifacts	All files below `outputs/`	Inspected as retained evidence; not overwritten or assigned new checkpoint provenance

The available repository contains separate detector and classifier experiments. It does not contain a completed full-image inference path connecting DBNet regions to ShuffleNetV2 and a final label-level decision. Recognition, if required, is a separate task. This distinction is essential when presenting the project architecture and planning the later demo.